

Variómetro para vuelo en parapente: Detector de Velocidad Vertical en Tiempo Real

Un **variómetro** para vuelo en parapente es un sistema que provee **feedback auditivo y visual** en tiempo real informando al piloto sobre su velocidad vertical, ya sea de ascenso o descenso. Está basado en la medición de cambios en la **presión atmosférica**, los cuales son inversamente proporcionales a los cambios en la **altitud** del piloto. El propósito de este sistema es facilitar la **detección de corrientes de aire ascendente** para el piloto, las cuales permiten ganar altura para luego planear largas distancias. El sistema contendrá un menú principal donde se podrán ajustar configuraciones o iniciar/frenar el “modo variómetro”. Para esto se utilizará un buzzer pasivo para emitir los sonidos necesarios, una pantalla LCD, un sensor de presión atmosférica y un encoder rotativo con pulsador para navegar el menú.

Objetivos del Proyecto

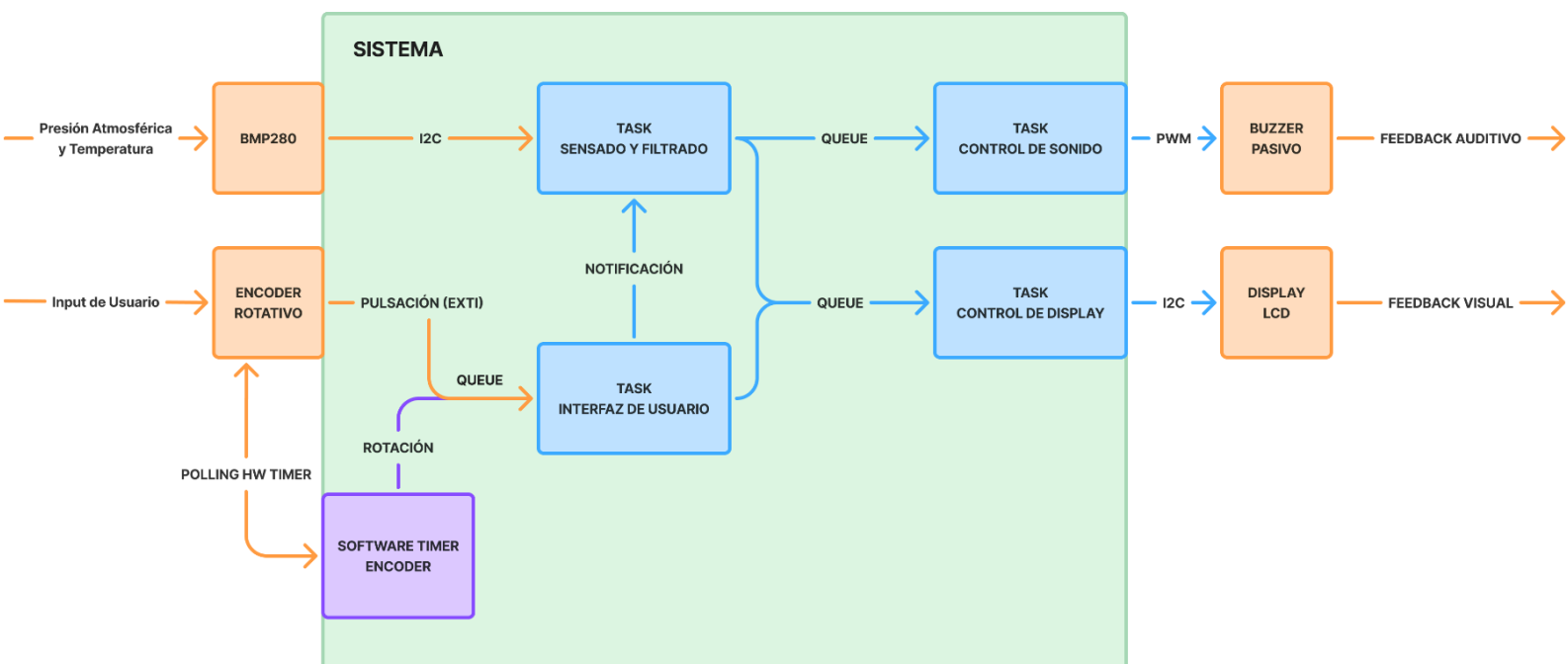
El objetivo principal del proyecto es desarrollar un sistema de variómetro para parapente con feedback visual y auditivo en tiempo real, configurable según las preferencias del piloto.

Objetivos específicos

- ❖ **Implementar un filtro de kalman** para la medición de presión y tasa de ascenso con parámetros ajustables.
- ❖ **Garantizar un tiempo de reacción** y de emisión de sonido acorde a los parámetros seleccionados por el usuario (hasta <200ms en el modo más sensible).
- ❖ **Desarrollar un sistema de feedback** auditivo capaz de variar dinámicamente la frecuencia y cadencia del sonido en función de la tasa de ascenso/descenso.
- ❖ **Construir una interfaz de usuario** interactiva para la visualización de datos de vuelo y un menú principal para la configuración de parámetros del sistema (tonos, umbrales, sensibilidad).

Descripción Funcional

Diagrama de Bloques



Entradas:

- ❖ **Sensor de Presión Atmosférica y Temperatura (BMP280):** Fuente de datos principal para el cálculo de altitud y velocidad vertical. Se lee a través de un bus de comunicación **I2C**.
- ❖ **Rotación del Encoder:** Utilizada por el usuario para la navegación del menú y configuración de parámetros. Se lee mediante un **Timer de hardware** configurado en modo **Encoder**.
- ❖ **Pulsador del Encoder:** Utilizado por el usuario para la selección de opciones y cambio de modos (Ej. salir del modo vuelo). Se detecta mediante una entrada **GPIO** configurada con interrupciones (**EXTI**).

Salidas:

- ❖ **Feedback Auditivo (Buzzer Pasivo):** Actuador principal del sistema que emite los tonos de ascenso/descenso en tiempo real. Es manejado mediante una salida de **PWM** generada por un **Timer de hardware** para controlar su frecuencia.
- ❖ **Interfaz de Usuario (Display LCD):** Interfaz para mostrar el menú de configuraciones y los datos de vuelo en tiempo real. Se actualiza mediante comandos enviados a través de un bus **I2C**.

Arquitectura y diseño en FreeRTOS

La única modificación de esta sección se encuentra en la parte de interrupciones, donde se agregó un software timer en modo one-shot que interactúa con las interrupciones del pulsador para detectar pulsaciones largas. El resto se mantiene igual, debido a que al momento de realizar el primer informe el proyecto ya estaba en una etapa avanzada.

Tareas Principales y Prioridades

- ❖ **Tarea de Interfaz de Usuario:**
 - Prioridad baja, se requiere tiempo de respuesta “humano”.
 - Responde a eventos de un encoder rotativo, permitiendo la navegación de un menú, cambios de configuraciones e inicio/frenado de vuelos.
- ❖ **Tarea de Sensado y Filtrado:**
 - Prioridad máxima para garantizar frecuencia de muestreo constante.
 - Lee los datos del sensor de presión atmosférica y aplica un filtro de Kalman a una frecuencia constante.
- ❖ **Tarea de Control de Sonido:**
 - Segunda prioridad más alta. Luego de que se produzca un dato por la tarea de sensado, lo primero que debe ocurrir es el feedback auditivo. No es prioridad máxima ya que si la tarea de sensado está leyendo un nuevo dato, conviene esperar antes de emitir un sonido que quizás ya no es relevante.

- Esta tarea emite los sonidos a distintas frecuencias y cadencias que indican ascenso/descenso al piloto a partir de los datos obtenidos del sensor.
- ❖ **Tarea de Control de Display:**
 - Prioridad mínima, tiempo de respuesta “humano”.
 - Esta tarea recibe eventos de actualización de menú o datos de vuelo y muestra lo requerido en el panel LCD.
- ❖ **Extra: Software Timer para detectar rotaciones del encoder**
 - Modo **AutoReload** con período de **50ms**
 - Asociado a una función de callback que lee el valor del timer asociado al encoder e informa al menú ante eventos de rotación.
 - Se puede **deshabilitar** en modo de vuelo para reducir jitter ya que las rotaciones sólo interesan en el menú principal y **rehabilitar** tras salir del modo de vuelo mediante una pulsación larga.

Sincronización y Comunicación

Una vez iniciado el “modo de vuelo”, las tareas de control de sonido y de control de display dependen exclusivamente de los datos generados por la tarea de sensado y filtrado (se bloquean esperando estos datos en una **queue**), y esta última se encargará de enviarles siempre el dato más reciente mediante el servicio **xQueueOverwrite**.

Se utilizará una **queue de eventos** a través de la cual se bloquea la tarea de interfaz de usuario hasta recibir eventos de rotación, pulsación larga o pulsación corta del encoder para luego procesarlos según corresponda.

La tarea de control de display recibirá distintos comandos mediante una **queue**. Cada comando puede tener datos asociados o no (prender, apagar, mostrar menú, mostrar datos de vuelo, etc) y estos comandos pueden ser enviados desde la tarea de interfaz de usuario o la tarea de sensado, dependiendo el estado del sistema (en vuelo o en menú).

Se utilizarán **Direct to Task Notifications** (o un semáforo binario) para el inicio/frenado de la tarea de sensado.

Interrupciones

La principal fuente de interrupciones es el pulsador del encoder, en donde se generan interrupciones de GPIO ante Falling (presionado) y Rising (liberado). A esto se le suma un **one-shot timer** que se lanza cuando el encoder es presionado y su callback es ejecutada sólo si pasa suficiente tiempo para ser considerado un “long press” antes de que sea liberado. La coordinación entre este timer y la interrupción que se genera al soltar el pulsador es manejada por un semáforo no bloqueante el cual asegura que sólo se genera un evento por pulsación (long press o click respectivamente). Esta forma de manejar las pulsaciones resultó ser mucho más intuitiva desde el punto de vista del usuario que la idea anterior de esperar a que se suelte para calcular la duración y luego decidir qué evento generar.

Grado de Avance

El desarrollo no está completamente finalizado por detalles menores, pero el sistema completo ya es funcional. El variómetro ya es completamente configurable a través de la interfaz de usuario y provee información de vuelo en tiempo real. En cuanto al código, tengo pensado seguir mejorando el manejo de errores y la calidad en general, pero ya están todas las tareas implementadas y funcionando como se esperaba en el diseño.

Todos los servicios de HAL para comunicación con periféricos ya fueron probados individualmente y luego integrados al proyecto.

Las pruebas generales se realizaron de dos formas. Primero se probaron los periféricos individualmente (sin tareas de FreeRTOS), verificando que la comunicación funcione de la manera que se espera (detectar dispositivos i2c, leer datos de presión atmosférica, mostrar texto en la LCD, detectar rotaciones del encoder, emitir sonidos a distintas frecuencias a través del buzzer pasivo). Una vez en funcionamiento, fueron integrados al proyecto (con tareas) y se probó el sistema completo simplemente usándolo y debuggeándolo. Cuando tenga la posibilidad, ajustaré los parámetros del sistema para intentar que el comportamiento sea lo más similar posible al de un variómetro profesional.

Faltaría que el sistema sea también un poco más robusto físicamente, aunque para los propósitos de la materia será armado a modo de prototipo.